Debugging Your Code

DATA 118 Lab — Student Version

Goals

The goal of this lab is to learn about common error messages and practice debugging your code.

Setup

For this lab, we will be using the `numpy` and `math` modules.

Exercise

```
import numpy as np
import math
```

What is a bug?

Debugging is the process of identifying and fixing 'bugs' in your code. A **bug** is computer science term for an error. This is thought have been popularized from "The 1947 Moth Incident", which involved a literal bug.

From the Smithsonian National Museum of American History:

> In 1947, engineers working on the Mark II computer at Harvard University found a moth stuck in one of the components. They taped the insect in their logbook and labeled it "first actual case of bug being found." The words "bug" and "debug" soon became a standard part of the language of computer programmers. Among those working on the Mark II in 1947 was mathematician and computer programmer Grace Hopper, who later became a Navy rear admiral. This log book was probably not Hopper's, but she and the rest of the Mark II team helped popularize the use of the term computer bug and the related phrase "debug."

IBM/Harvard University (1947). Log Book With Computer Bug [Photograph]. Smithsonian National Museum of American History. https://americanhistory.si.edu/collections/object/nmah_334663

Bugs may either

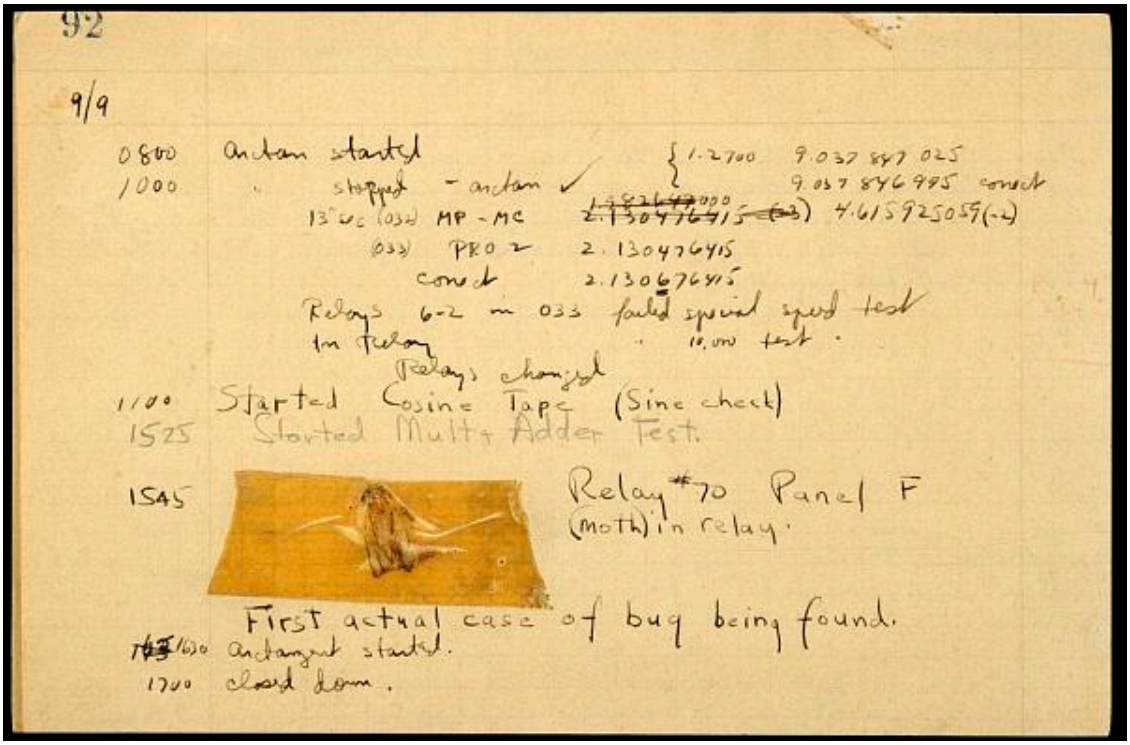


Figure 1: Log Book With Computer Bug [Photograph]

- Cause your code to produce an error message, or
- Cause your code to produce incorrect results (**logical errors**).

Error Messages

There are many specific types of error messages you might encounter when using Python that can generally be sorted into two main types:

- **Syntax Errors** - made before execution
  - `SyntaxError`
- **Exceptions** - made during execution
  - `NameError`
  - `IndexError`
  - `AttributeError`
  - `KeyError`
  - `ImportError`
  - `TypeError`
  - `ValueError`
  - `ModuleNotFoundError`
  - ...

When one of these errors occurs, Python will return an error message. We are very used to thinking of errors as a bad thing, but while programming, you may come to actually appreciate error messages, as they can help you go back and rewrite your code to run correctly.

Syntax Errors

If your code includes a syntax error, you will see a message starting with `SyntaxError`.

These messages typically include:

- A file name or cell input number,
- A line number, and
- An arrow (or arrows) pointing to the issue.

The issue could be that something is missing…

Exercise

```
for i in [1,2]
    print(i+8)
```

or that something was written incorrectly.

Exercise

```
for i in [1,2],
    print(i+8)
```

Sometimes, the software you are using will try to warn you about syntax errors ahead of time using red or yellow lines underneath problematic parts of your code. This does not show up in this lab setup, but if you copy and past the code below into an IDE like Jupyter or VSCode, you will see these lines.

Exercise

```
mean([1,2,3,4)
```

These errors occur any time we make a mistake with regards to the rules that define how Python code is written. For example, when Python sees a `for`-loop, it expects to see a colon `:`. When it sees an open bracket `[`, it expects to see a closed bracket `]`. If your syntax is not correct, Python does not execute the code. Once your code is free of syntax errors, it will be executed.

Sometimes you may get special types of `SyntaxErrors`. A common one is the `IndentationError`, which occurs when you don't have the proper indentation in a for-loop, function, etc

Exercise

```
for i in [1,2]:
print(i+1)
```

Other times, the `SyntaxErrors` can be very vague and unhelpful. It can be useful to use the line numbers to identify where the error is, even if you don't know exactly what the error is. Then, we recommend looking closely at the various "punctuation marks" to see what is missing or incorrect.

Exceptions

During code execution, you could receive other error messages known as exceptions. You will see a message starting with the exception type (e.g., a `NameError`).

Typically includes:

- A file name or cell input number,
- A line number,
- A brief explanation, and
- **traceback** - the context in which the exception occurred
  - Usually in the form of a few lines of code.
  - More common when you write functions.

The exception type should give you clues as to what went wrong.

Note:
Because we are working in a Shiny environment there is also a line that says "Run reticulate::py_last_error() for details." You may ignore this, it is not relevant to you.

Exercise

```
print(my_string)
```

Exercise

```
5 + "6"
```

Exercise

```
a = 1 + 2
c = a + b
```

Let's walk through some exception types and what they mean...

We've already seen `NameError` and `TypeError`.

- `NameErrors` occur when you try to use a variable name that hasn't been assigned yet.
  - This is very common when running cells in your Jupyter notebook out of order or when you've restarted your Kernel

- `TypeErrors` occur when you use the wrong data type for whatever function or operand you are using.
  - These are common when you don't know the datatype of a `DataFrame` column (i.e., `"2016"` looks like `2016`).
  - When addressing `TypeErrors`, it can be useful to use functions like `type()` or `dtype()`:
    * `type()` gives you the data type but, if used on a collection of data, will only give you the type of container (ie `array`, `Series`, ...).
    * `dtype()` gives you the data type of the *elements* inside the container

Exercise

```
print(type("2016"), type(np.array([1, 2, 3])), np.array([1, 2,
↪  3]).dtype)
```

Lookup Errors

`IndexErrors` and `KeyErrors` are both types of exceptions called `LookupErrors`. They occur when you try to access a dictionary with a key that is invalid, use an invalid index for an array or DataFrame, etc.

`IndexErrors` and `KeyErrors` commonly occur when:

- You use `.groupby()` or otherwise manipulate a Data Frame and forget you have a new index.
- You forget Python indexes start at `0`.
- You just mistype the key or index.

Exercise

```
my_dict = {"a":1, "b":2, "c":3}
my_dict[1]
```

Exercise

```
my_list = [1,2,3]
my_list[3]
```

Other Common Error Types

`AttributeErrors` occur when there has been a failure to set or access an attribute for some object. `AttributeErrors` commonnly occur when the object you're working with does not actually have that attribute–don't forget that attributes are characteristics specific to a particular type of object (like a list, or a data frame, etc.). They can also happen if you forget parentheses on a method–first, check to make sure you've

included all of the necessary characters, then, check to make sure your object is the type that you think it is.

Exercise

```
my_list.size
```

Exercise

```
my_list.mean
```

`ValueErrors` occur when an invalid value is given to a function or operand. These are common when using mathematical functions.

Exercise

```
import math
math.sqrt(-1)
```

Exercise

```
float("number")
```

These all come from base Python, but imported modules can have their own error types as well! You may encounter more as you learn about more modules and more programming techniques, but the error types we've covered should give you a pretty solid basis for debugging.

Logical Errors

Sometimes your code does not encounter an exception, but you don't get the output that you expected. This is very common, especially when writing your own functions! Unfortunately, Python won't return an error message here–it is up to you to test and examine your code until you are comfortable with the output it is producing.

Here is an example:

Exercise

```
def my_min(my_array):
    '''A function that takes in an array of numbers and returns the
    ↪  smallest number'''
    smallest = 10
    for number in my_array:
        if number < smallest:
            smallest = number
    return smallest

my_min(np.array([6,3,2,5]))
```

Exercise

```
my_min(np.array([23,47,13,78]))
```

This function isn't giving me the answer I would expect... I need to look at the code inside my function to see what is wrong.

```
smallest = 10
for number in my_array:
    if number < smallest:
        smallest = number
```

It can be useful to do a few things when trying to debug code:

1) Think through the logic without worrying about the code itself. Should this approach work?
   - Sometimes we write 'pseudocode' or draw diagrams
2) Add print statements to see what is happening throughout the code
   - Where are things changing?
   - It works as expected until which line?
   - If you are using conditionals, is the condition ever met?
   - If you are using loops, is the loop working the way you expect?

Exercise

```
def my_min(my_array):
    '''A function that takes in an array of numbers and returns the
    ↪  smallest number'''
    smallest = 10
    for number in my_array:
        print("The number is: ",number)
        print("The smallest so far is: ",smallest)
        if number < smallest:
            print("I made it into the if-statement")
            smallest = number
    return smallest

my_min(np.array([23,47,13,78]))
```

From my print statements, I can tell

- The loop is iterating over the array correctly,
- The smallest number is always 10, and
- The condition for my `if`-statement was never met.

10 is smaller than all of the numbers in my array so `smallest` never gets set to anything else! We should initialize `smallest` to be something much larger than 10.

Exercise

```
def my_min(my_array):
    '''A function that takes in an array of numbers and returns the
    ↪  smallest number'''
    smallest = math.inf
    for number in my_array:
        if number < smallest:
            smallest = number
    return smallest

my_min(np.array([23,47,13,78]))
```

Error Types Activity

Each of the code chunks below has code that would raise an error. Choose the name of the error that would be raised in the chunk. Discuss with your neighbors why these errors might appear! Then, select a possible fix.

For example,

```
15/0
```

The code in this chunk would raise a `ZeroDivision` error because you cannot divide

by zero. To fix it, you might replace the zero with some other integer or float value.

```
state_capitals = {"Illinois": "Springfield",
                  "Massachusetts": "Boston",
                  "Texas": "Austin",
                  "Virginia": "Richmond"}

state_capitals["Washington"]
```

Question 1. What kind of error will be raised in the code chunk above?

☐ `AttributeError`
☐ `IndexError`
☐ `KeyError`
☐ `NameError`
☐ `SyntaxError`
☐ `TypeError`
☐ `ValueError`
☐ `ZeroDivisionError`

Discuss with a neighbor (or on Ed):
Why does this code raise that kind of error?

Question 2. How would you fix this error?
Answer:
_____

```
state_capitals["Virginia"]]
```

Question 3. What kind of error will be raised in the code chunk above?

☐ `AttributeError`
☐ `IndexError`
☐ `KeyError`
☐ `NameError`
☐ `SyntaxError`
☐ `TypeError`
☐ `ValueError`
☐ `ZeroDivisionError`

Discuss with a neighbor (or on Ed):
Why does this code raise that kind of error?

Question 4. How would you fix this error?
☐ Remove the second close bracket (`]`).
☐ Add a second open bracket (`[`).
☐ Replace the brackets with parentheses (`()`), because `state_capitals` should be a function.
☐ Add a comma after `Virginia`, because we are indexing by rows.

```
states = 50
print("There are " + states + " in the US.")
```

Question 5. What kind of error will be raised in the code chunk above?
☐ `AttributeError`
☐ `IndexError`
☐ `KeyError`
☐ `NameError`
☐ `SyntaxError`
☐ `TypeError`
☐ `ValueError`
☐ `ZeroDivisionError`

Discuss with a neighbor (or on Ed):
Why does this code raise that kind of error?

Question 6. How would you fix this error?
☐ Convert `states` to a string using `str()`.
☐ Use an operator other than the plus sign (`+`) for correct string/integer concatenation.
☐ Rewrite the entire phrase as a string, there is no need to concatenate three separate pieces.
☐ Convert `There are` and `in the US` to be integers using `int()`.

```
capitals_data = {'Capital': ["Springfield", "Boston", "Austin",
 ↪  "Richmond"]}
capitals_df = pd.DataFrame.from_dict(capitals_data)
capitals_df.index = ["Illinois", "Massachusetts", "Texas", "Virginia"]

capitals_df.iloc[50,]
```

Question 7. What kind of error will be raised in the code chunk above?
☐ `AttributeError`
☐ `IndexError`
☐ `KeyError`
☐ `NameError`
☐ `SyntaxError`
☐ `TypeError`
☐ `ValueError`
☐ `ZeroDivisionError`

Discuss with a neighbor (or on Ed):
Why does this code raise that kind of error?

Question 8. How would you fix this error?
☐ Change the index (`50`) to one of `0`, `1`, `2`, or `3`.
☐ Change the index, `50`, to `49`, because Python uses zero-based indexing.
☐ Change `.iloc` to `.loc`.
☐ Use `orient = index` in `to_dict()`.

```
capitals_df.len()
```

Question 9. What kind of error will be raised in the code chunk above?
☐ `AttributeError`
☐ `IndexError`
☐ `KeyError`
☐ `NameError`
☐ `SyntaxError`
☐ `TypeError`
☐ `ValueError`
☐ `ZeroDivisionError`

Discuss with a neighbor (or on Ed):
Why does this code raise that kind of error?

Question 10. How would you fix this error?
☐ Use `capitals_df.shape`, because `.shape` is the attribute of the DataFrame that contains the dimensions.
☐ Use `capitals_df.shape()`, because `.shape()` is the method for a DataFrame that will calculate the dimensions.
☐ Use `len(capitals_df)`, because `len()` is a function and not a method.
☐ Use `capitals_df.length()`, because `.len()` is not the right method name.

Debugging Activity

Now, you try! Find and fix the syntax errors, exceptions, and logical errors in the code blocks below.

Debug this combination of shapes and colors. Recall that a standard indent in Python is 4 spaces.

Exercise

```
shapes = ["circle", "square", "rectangle", "triangle", "rhombus",]
colors = ["red", "green", "blue"]
for i in range(5):
for j in range(3):
    print("A group of " + colors[i] + " " + shape[j] + "s")
```

Debug this set of factorials.

Exercise

```
number = 5
 factorial = 1

for i in range(1, number);
    factorial = factorial * "i"

print(f"The factorial of {number} is {factorial")
```

Debug this sum of even numbers.

Exercise

```
numbers = [1 2, 3, 4, 5, 6, 7, 8, 9, 10]
even_sum = 0

for nums in numbers
    if num % 2 == 0
        even_sum = even_sum + num
    else
        even_sum = even_sum - num

print(f"The sum of all even numbers in the list is: {even_sum}")
```